HCMC University of Technology, VNU



Software Architecture

Chapter 5. Architecture Styles

Lecturer: Thai-Minh Truong, PhD.

Email: thaiminh@hcmut.edu.vn

CHAPTER 5. Architecture Styles



5.1. Fundamental Patterns

5.2. Monolithic Versus Distributed Architectures

5.3. Layered Architecture Style

5.4. Pipeline Architecture Style

5.5. Microkernel Architecture Style

5.1. Fundamental Patterns



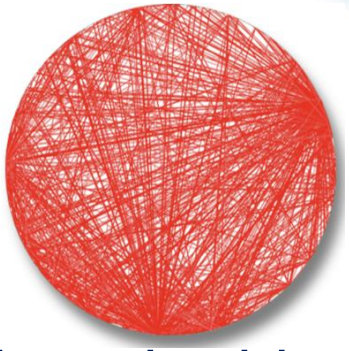
- **Architecture style**: the overarching structure of how the user interface and backend source code are organized (such as within layers of a monolithic deployment or separately deployed services) and how that source code interacts with a datastore.
- **Architecture patterns:** on the other hand, are lower-level design structures that help form specific solutions within an architecture style (such as how to achieve high scalability or high performance within a set of operations or between sets of services).

=> Several fundamental patterns appear again and again throughout the history of software architecture because they provide a useful perspective on organizing code, deployments, or other aspects of architecture.

5.1. Fundamental Patterns



“Big Ball of Mud”



- "Big Ball of Mud" signifies chaotic software structures lacking clear architecture.
- Such systems suffer from haphazard growth and ad hoc fixes, causing difficulties in management and scalability.
- Architects avoid this pattern due to its hindrance to development and maintenance.
- Unintentional creation often results from a lack of governance around code quality.

5.1. Fundamental Patterns



Unitary Architecture

- Unitary Architecture: Initially, software and hardware were unified.
- Mainframes and early personal computers operated as single systems.
- With technological advancement, data separation and distributed systems emerged.
- Unitary architectures are now rare, as software expands, necessitating separation for performance and scalability.

5.1. Fundamental Patterns



- **Client/Server**

Over time, various **forces required partitioning** away from a single system; how to do that forms the basis for many of these styles. A **fundamental style** in architecture **separates technical functionality** between **frontend** and **backend**, called a two-tier, or client/server, architecture.

- **Desktop + database server**

Early PC architectures favored rich desktop apps with separate database servers, leveraging standalone servers via standard network protocols for intensive tasks.

5.1. Fundamental Patterns



- **Browser + web server**

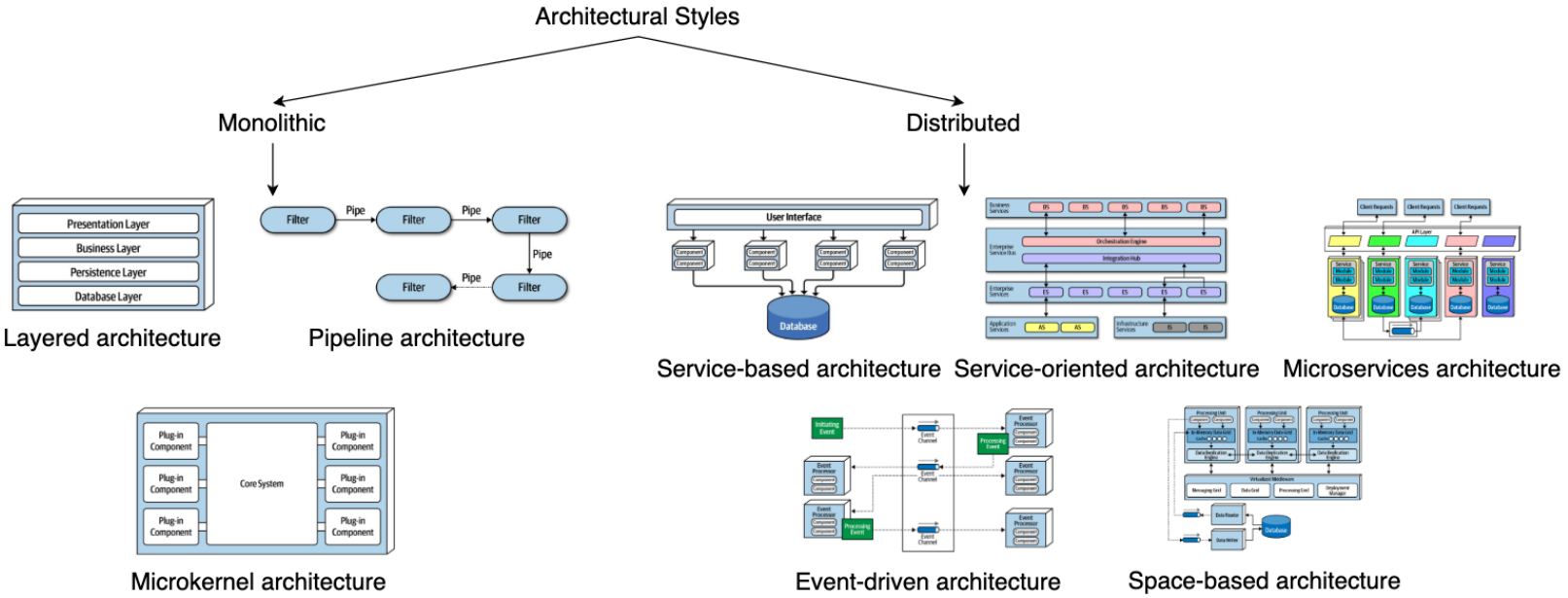
In modern web development, browsers connect to web servers, which in turn access separate database servers, often termed as a two-tier architecture.

- **Three-tier**

In the late 1990s, the three-tier architecture gained popularity, simplifying distributed system development with robust database tiers, application tiers, and frontends.

5.2. Monolithic Versus Distributed Architectures

- Architecture styles can be classified into two main types:
  - ***Monolithic*** (single deployment unit of all code)
  - ***Distributed*** (multiple deployment units connected through remote access protocols).



5.2. Monolithic Versus Distributed Architectures



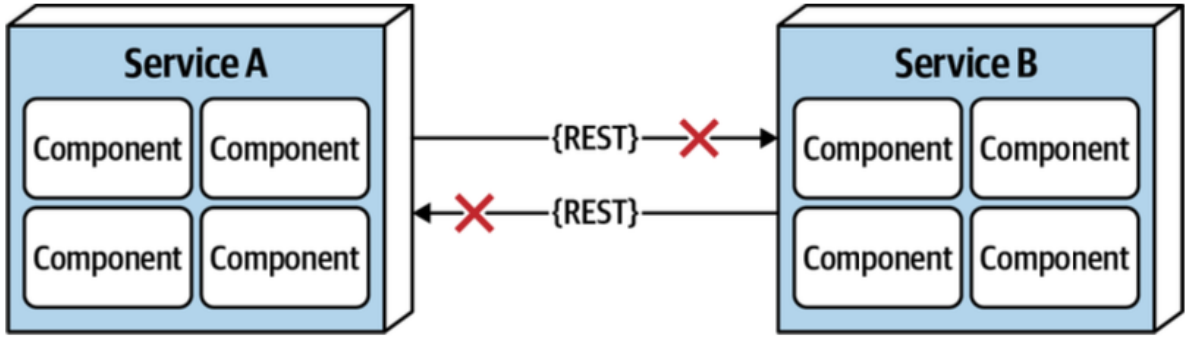
- Distributed architecture styles, while being much more powerful in terms of performance, scalability, and availability than monolithic architecture styles, have significant trade-offs for this power.
- 8 fallacies (issues) of distributed computing apply to distributed architectures today:

5.2. Monolithic Versus Distributed Architectures



Fallacy #1: The Network Is Reliable

Network reliability remains generally unreliable, impacting distributed architectures and necessitating measures like timeouts and circuit breakers, particularly evident in microservices.

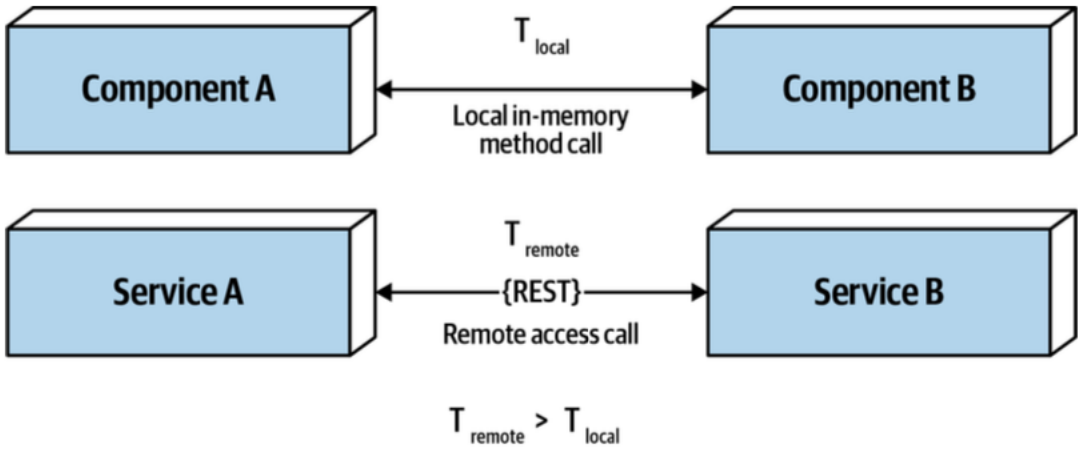


5.2. Monolithic Versus Distributed Architectures



Fallacy #2: Latency Is Zero

Architects must recognize the latency difference between local and remote calls, crucial in assessing feasibility and performance in distributed architectures like microservices.

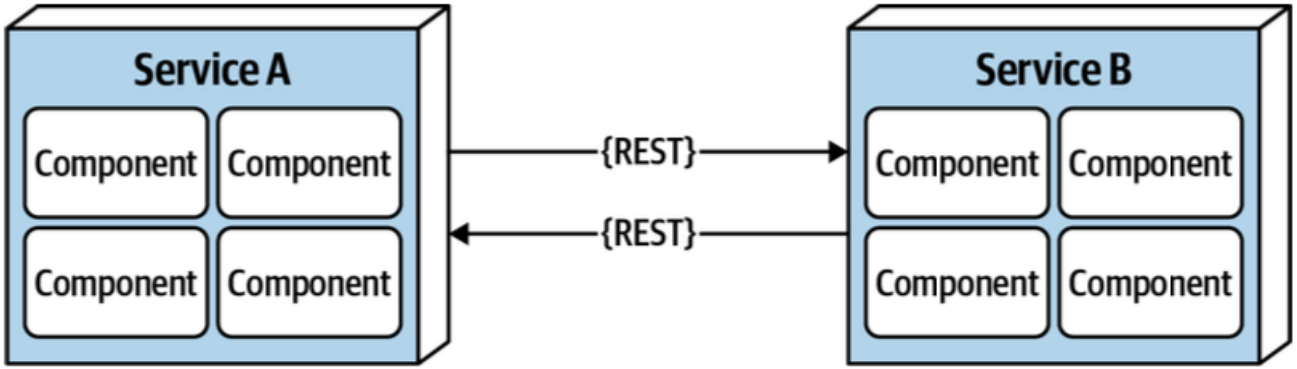


5.2. Monolithic Versus Distributed Architectures



Fallacy #3: Bandwidth Is Infinite

Breaking systems into smaller units increases bandwidth usage between services, highlighting the need to minimize data transmission for optimal performance

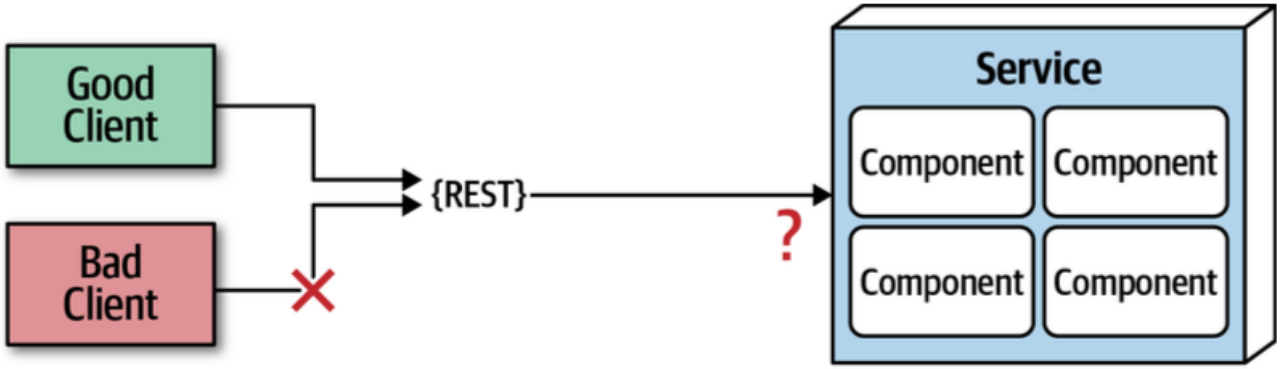


5.2. Monolithic Versus Distributed Architectures



Fallacy #4: The Network Is Secure

Most architects and developers get so comfortable using virtual private networks (VPNs), trusted networks, and firewalls that they tend to forget about this fallacy of distributed computing: the network is not secure.

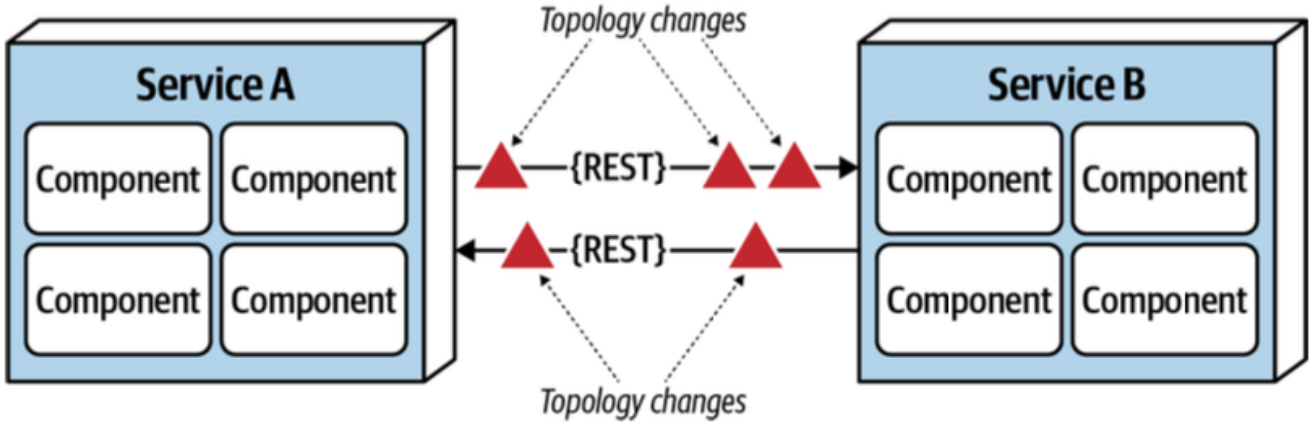


5.2. Monolithic Versus Distributed Architectures



Fallacy #5: The Topology Never Changes

Assuming fixed network topology can cause disruptions like service timeouts, highlighting the need for architects to stay in constant communication with operations for effective adaptation.



5.2. Monolithic Versus Distributed Architectures



Fallacy #6: There Is Only One Administrator

Architects overlook the complexity of coordinating with multiple network administrators, complicating issue resolution in distributed architectures compared to monolithic applications.



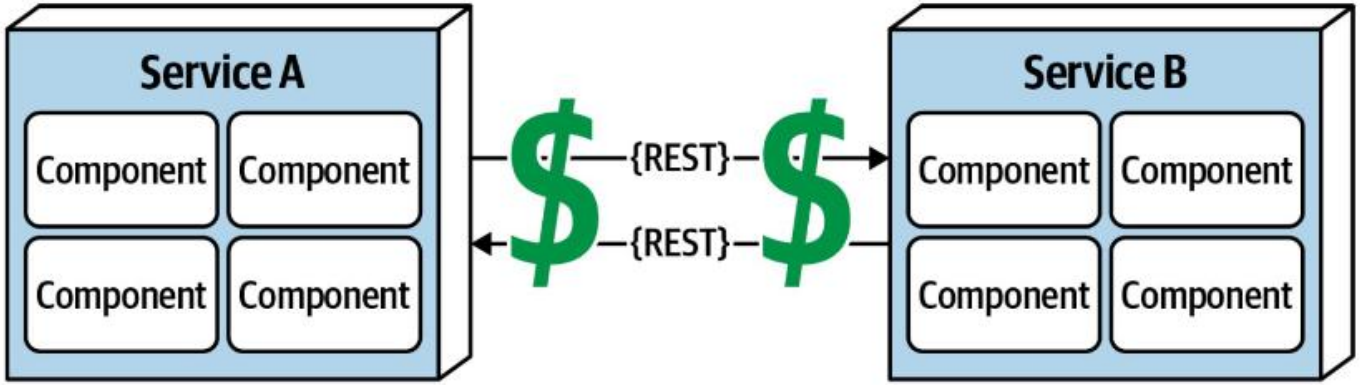
There are many network administrators, not just one

5.2. Monolithic Versus Distributed Architectures



Fallacy #7: Transport Cost Is Zero

Architects confuse transport cost with latency in distributed architectures, emphasizing the need for thorough analysis of server and network topology.

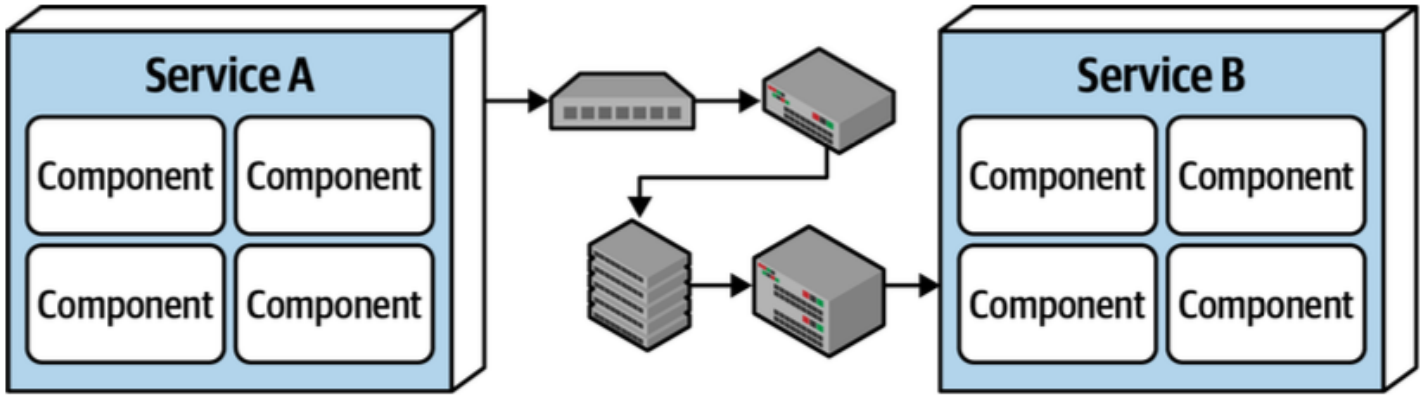


5.2. Monolithic Versus Distributed Architectures



Fallacy #8: The Network Is Homogeneous

Assuming network homogeneity overlooks integration challenges and packet loss from multiple vendors, impacting reliability and latency in distributed architectures, perpetuating frustration.



5.2. Monolithic Versus Distributed Architectures



Other consideration

- **Distributed logging**

In distributed architectures, scattered logs complicate root-cause analysis compared to monolithic applications, where a single log simplifies the process, highlighting challenges with logs dispersed across various locations and formats.

- **Distributed transactions**

In monolithic architectures, transactions are straightforward with ACID guarantees, but in distributed architectures, eventual consistency sacrifices data integrity for scalability and performance (ACID vs BASE).

- **Contract maintenance and versioning**

In distributed architectures, contract creation, maintenance, and versioning pose challenges due to decoupled services owned by different teams, requiring complex communication for version deprecation.

5.2. Monolithic Versus Distributed Architectures



Exercises:

Search & discuss solutions for Distributed logging, Distributed transactions and Contract maintenance & versioning?

5.3. Layered Architecture Style



Topology

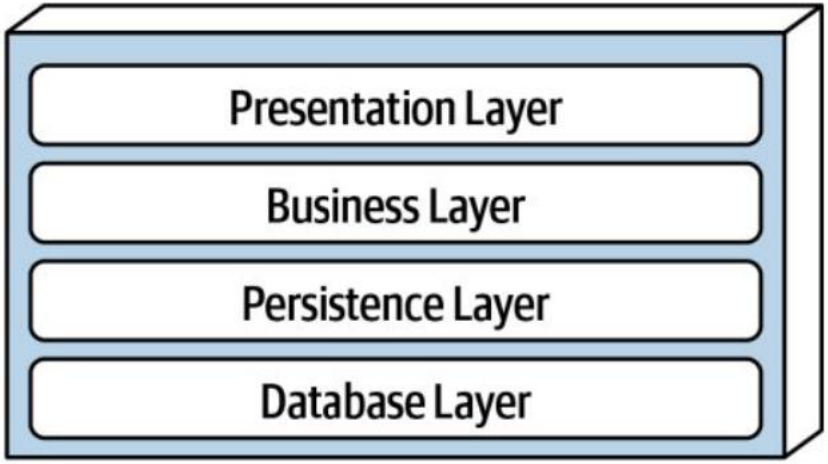
- **Organizational Structure:** Components in layered architecture are structured into logical horizontal layers.
- **Role Specificity:** Each layer fulfils a distinct role within the application, such as managing presentation or business logic.
- **Standard Layers:** While the number and types of layers can vary, most layered architectures incorporate four standard layers:
  1. Presentation
  2. Business
  3. Persistence
  4. Database
- **Flexibility:** No strict restrictions on the number or types of layers, allowing adaptation based on application requirements.

5.3. Layered Architecture Style



Topology

- Components organized into logical horizontal layers.
- Each layer has a specific role (presentation, business, persistence, database).
- Variability in the number of layers based on application complexity.

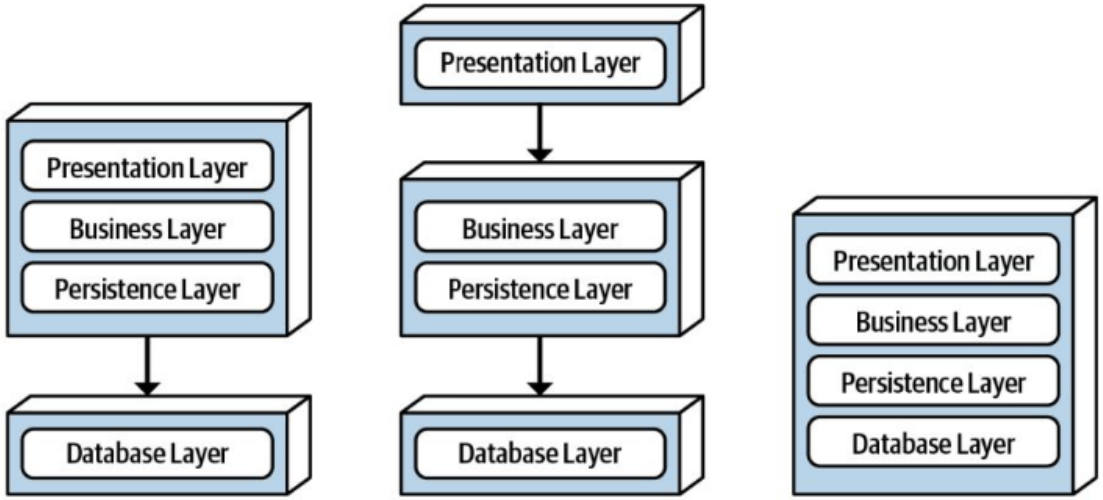


5.3. Layered Architecture Style



Topology variants

- **Variant 1:** Combines presentation, business, and persistence layers into one unit, with the database layer typically represented as a separate external physical database (or filesystem).
- **Variant 2:** Separates presentation into its unit, with business and persistence combined, the database layer is usually physically separated through an external database or filesystem.
- **Variant 3:** Combines all four layers into one deployment, including the database layer.



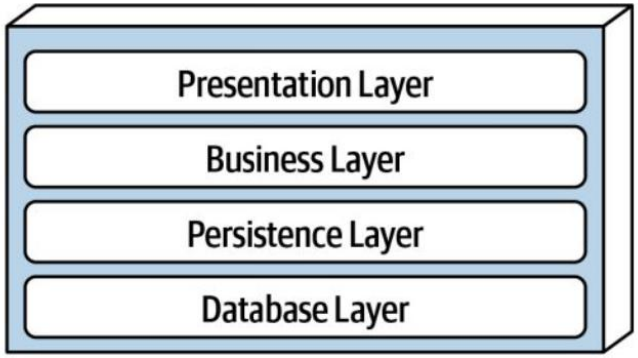
5.3. Layered Architecture Style



Topology

Separation of Concerns & Responsibilities of Each Layer

- Each layer handles specific tasks, forming an abstraction.
- Presentation layer: User interface and browser communication logic.
- Business layer: Executes business rules associated with requests.
- Persistence layer: Manages data access and storage.

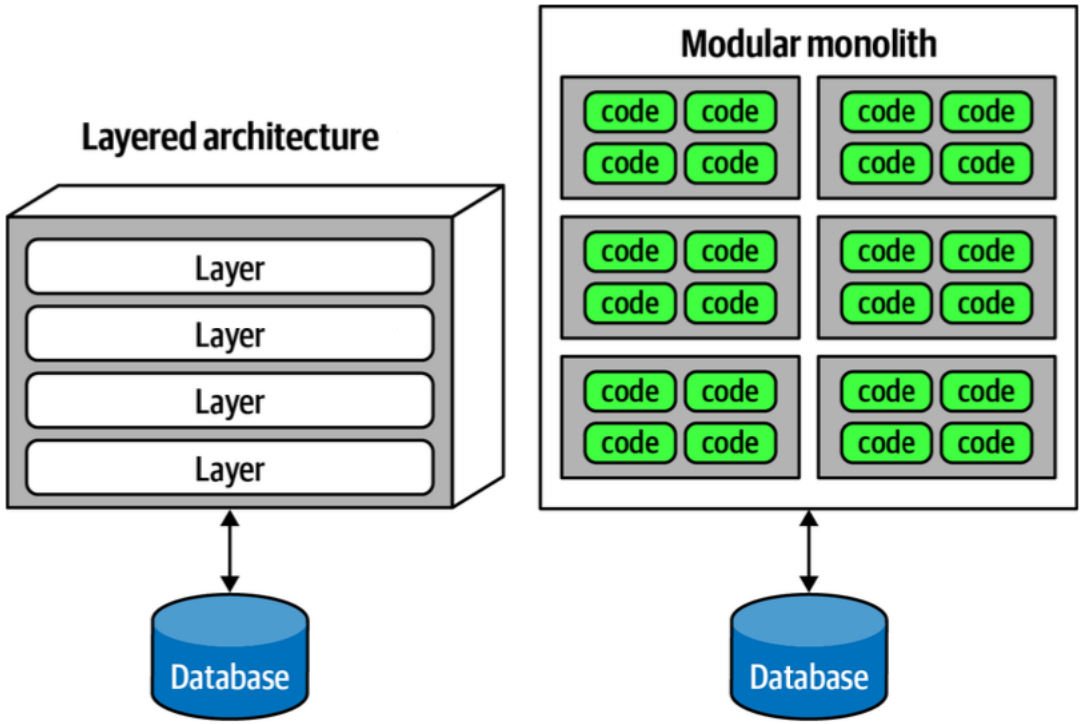


5.3. Layered Architecture Style



Topology

In-class activity: Compare two types of top-level architecture partitioning: layered and modular



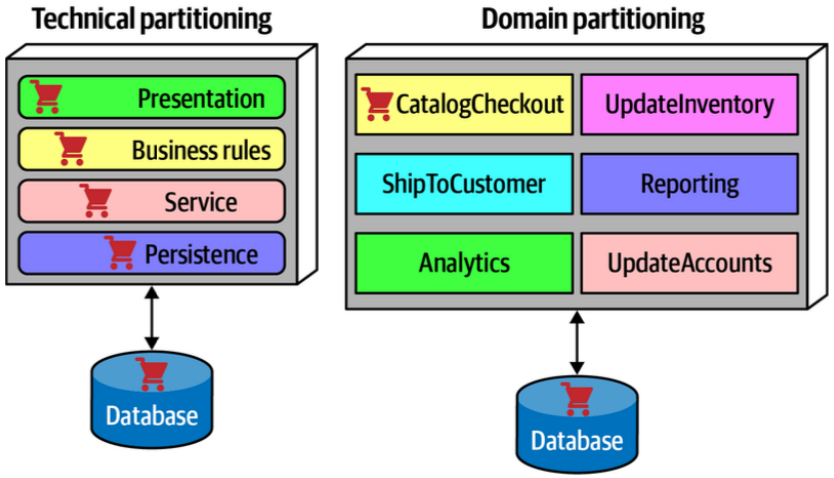
5.3. Layered Architecture Style



Topology

Technical vs. Domain Partitioning

- Layered architecture is technically partitioned.
- Components grouped by technical role (presentation, business) rather than domain.
- Domains spread across layers, challenging domain-driven design.



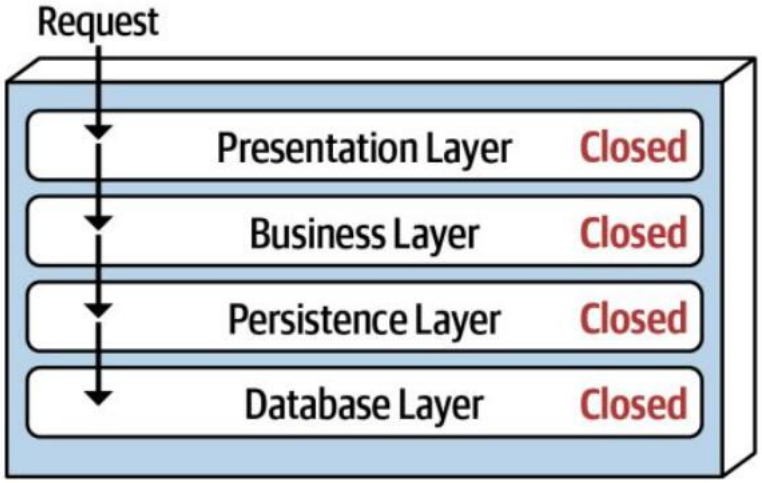
5.3. Layered Architecture Style



Layers of Isolation

Each layer in the layered architecture style can be either **closed** or **open**.

A closed layer means that as a request moves top-down from layer to layer, the request cannot skip any layers, but rather must go through the layer immediately below it to get to the next layer.



5.3. Layered Architecture Style



Layers of Isolation

- **Layers of Isolation Concept:**

◇ Changes in one layer ideally shouldn't impact others, maintaining layer independence.

◇ Each layer operates with minimal knowledge of others.

- **Importance of Closed Layers:**

◇ Necessary for maintaining layers of isolation.

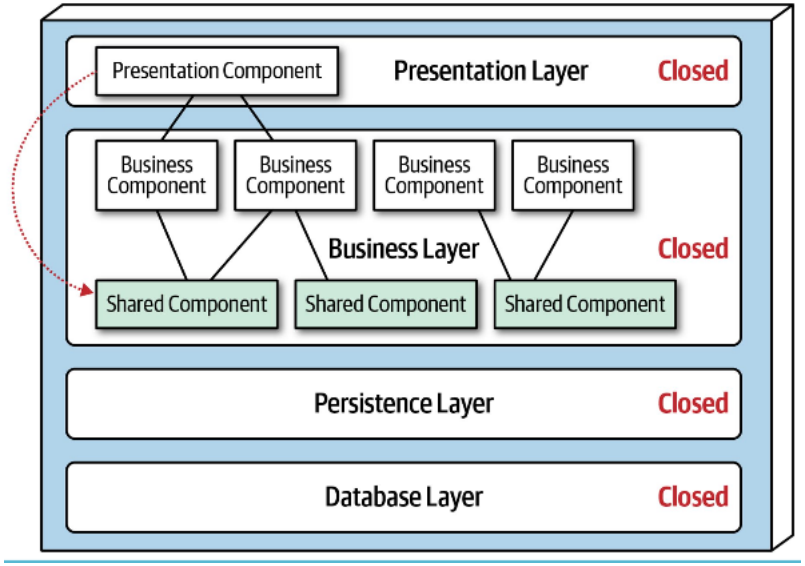
◇ Prevents tight coupling and interdependencies between layers.

◇ Enhances flexibility and ease of change.

5.3. Layered Architecture Style



Adding layers

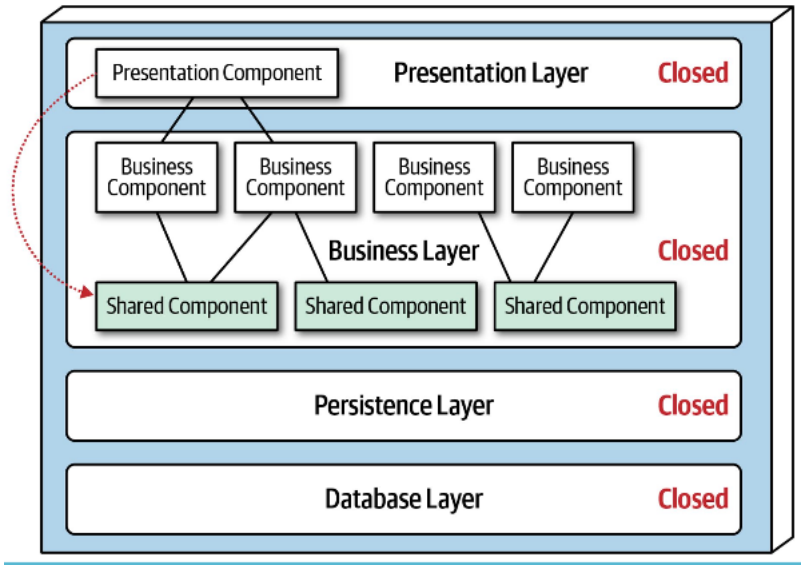


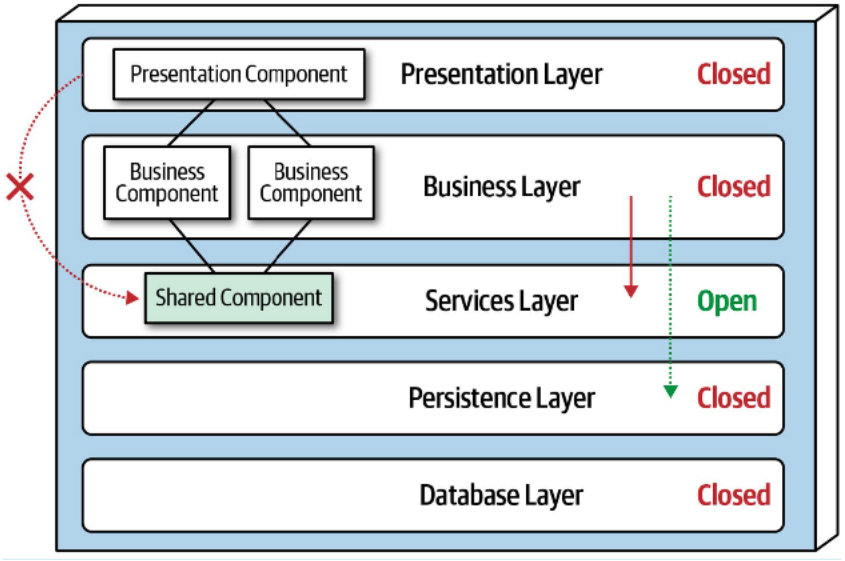
Shared objects within the business layer

5.3. Layered Architecture Style



Adding layers





Shared objects within the business layer

Adding a new services layer to the architecture

<u>In-class activity:</u> Compare two above architectures?

5.3. Layered Architecture Style



Other consideration

- **Architecture sinkhole anti-pattern:** occurs when requests move from layer to layer as simple pass-through processing with no business logic performed within each layer.
  - Eg.: The presentation layer responds to a simple request from the user to retrieve basic customer data (such as name and address). The presentation layer passes the request to the business layer, which does nothing but pass the request on to the persistence layer, which then makes a simple SQL call to the database layer to retrieve the customer data.
  - Every layered architecture will have at least some scenarios like this =>The 80-20 rule is usually a good practice to follow. For example, it is acceptable if only 20 percent of the requests are sinkholes. However, if 80 percent of the requests are sinkholes, it a good indicator that the layered architecture is not the correct architecture style for the problem domain.

5.3. Layered Architecture Style



Architecture Characteristics Ratings

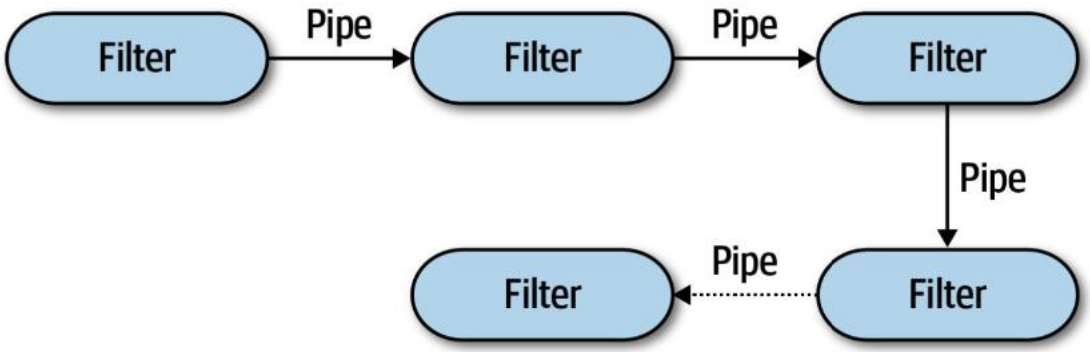
Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★

5.4. Pipeline Architecture Style



Topology

- The pipes and filters coordinate in a specific fashion, with pipes forming one-way communication between filters, usually in a point-to-point fashion.



5.4. Pipeline Architecture Style



Pipes

- Pipes in this architecture form the communication channel between filters.
- Each pipe is typically unidirectional and point-to-point (rather than broadcast) for performance reasons, accepting input from one source and always directing output to another.
- The payload carried on the pipes may be any data format, but architects favor smaller amounts of data to enable high performance.

5.4. Pipeline Architecture Style



Filter

- **Producer**

The starting point of a process, outbound only, sometimes called the source.

- **Transformer**

Recognized by functional advocates as "map," accepts input, optionally transforms data, and forwards it to the outbound pipe.

- **Tester**

Accepts input, tests one or more criteria, then optionally produces output, based on the test. Functional programmers will recognize this as similar to reduce.

- **Consumer**

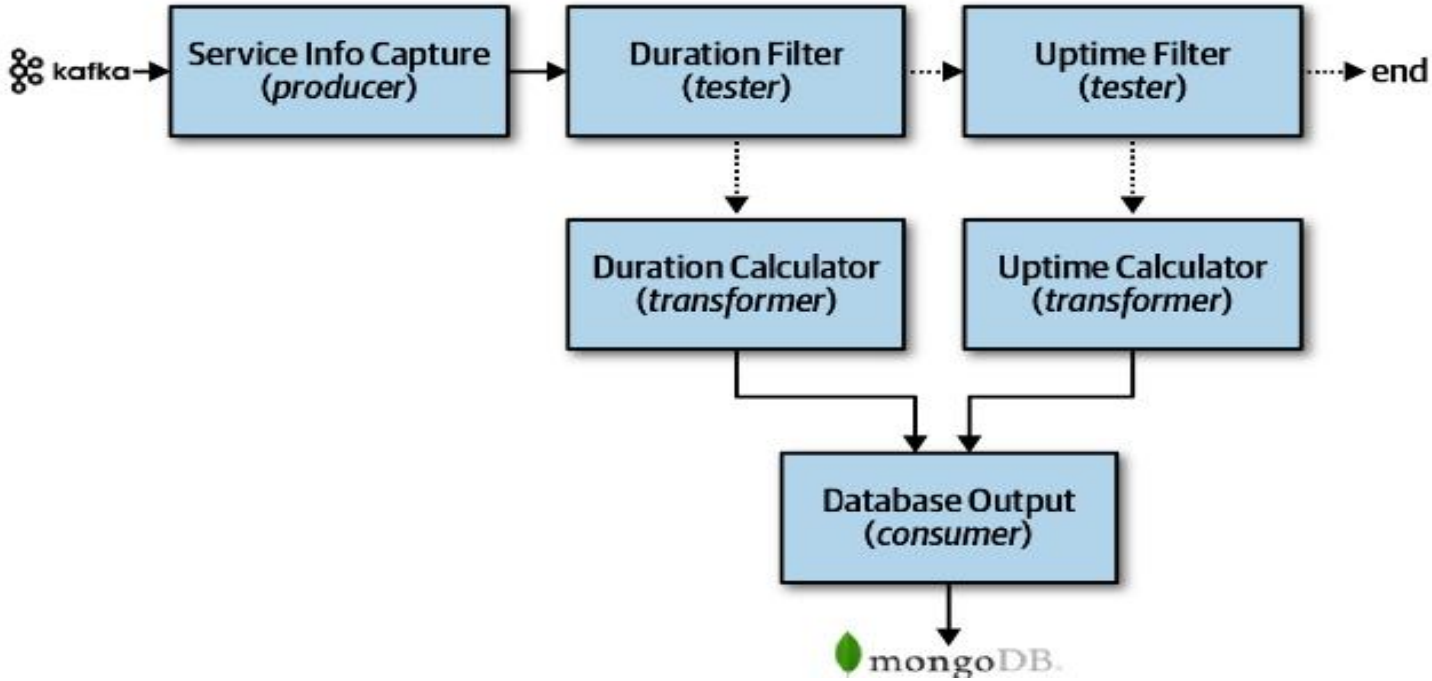
Consumers, serving as the termination point for pipeline flow, may persist the final result to a database or display it on a user interface screen.

5.4. Pipeline Architecture Style



Example

- Unix terminal shell languages, such as Bash and Zsh.
- Tools that utilize the MapReduce programming model.



Streaming to Apache Kafka

5.4. Pipeline Architecture Style



Architecture Characteristics Ratings

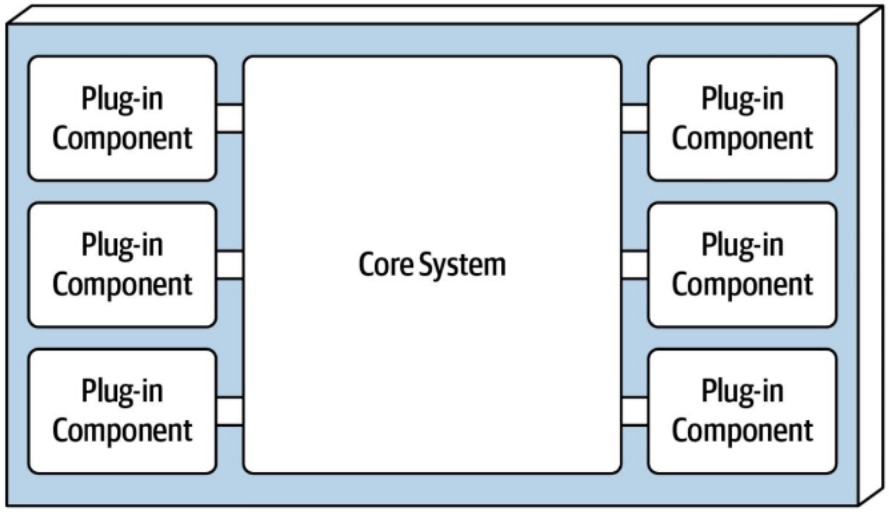
Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★★

5.5. Microkernel Architecture Style



Topology

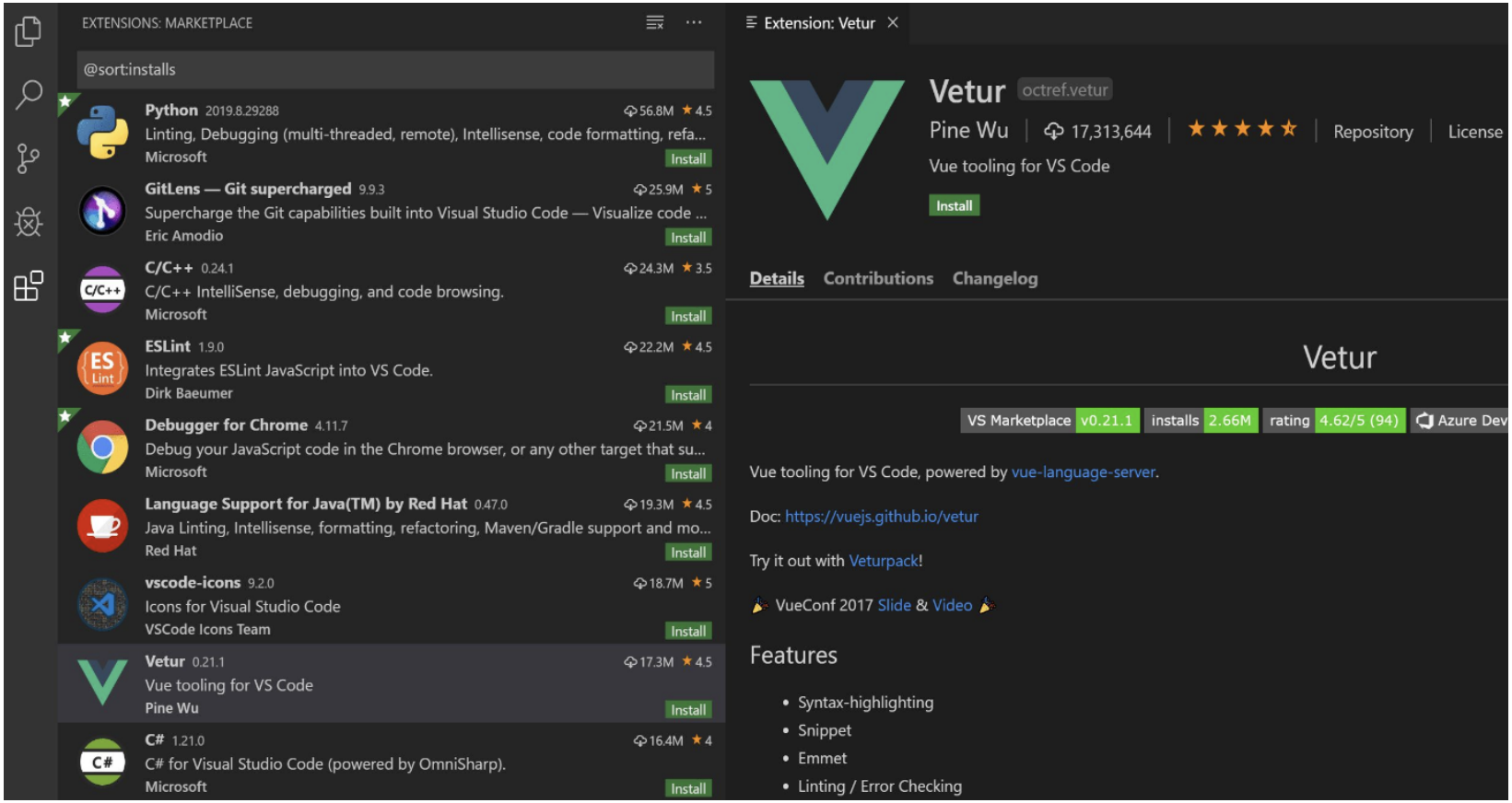
- The microkernel architecture style is a relatively simple monolithic architecture consisting of two architecture components: a core system and plug-in components (also referred to as the plug-in architecture).
- Providing extensibility, adaptability, and isolation of application features and custom processing logic.



5.5. Microkernel Architecture Style



Example

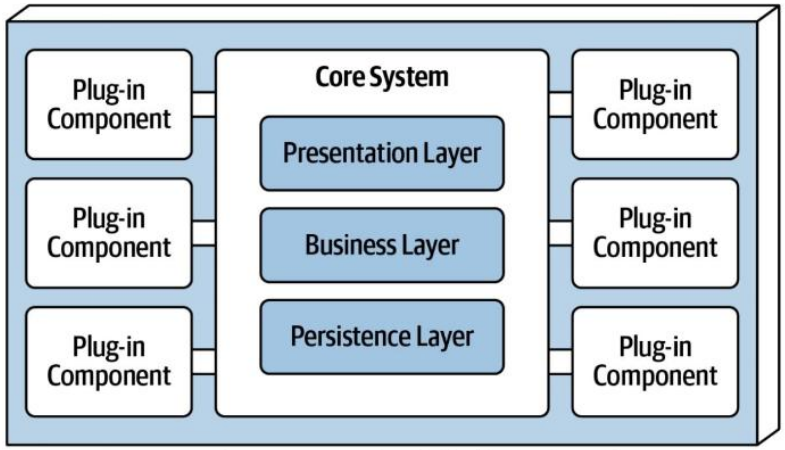


5.5. Microkernel Architecture Style

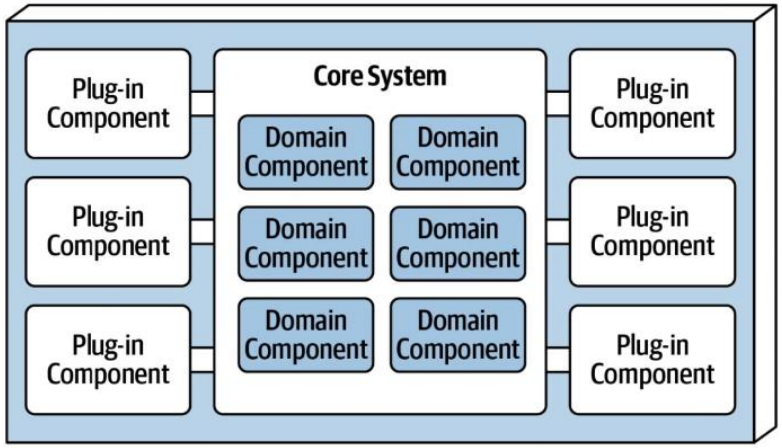


Core System

- Depending on the size and complexity, the core system can be implemented as:
  - A layered architecture or
  - Separately deployed domain services



Layered Core System (Technically Partioned)



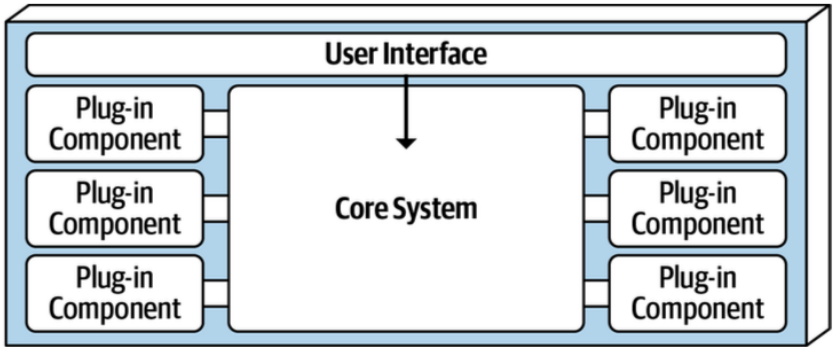
Modular Core System (Domain Partioned)

5.5. Microkernel Architecture Style

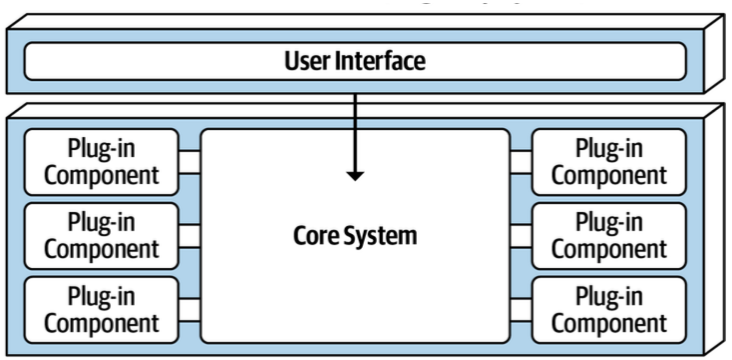


Core System

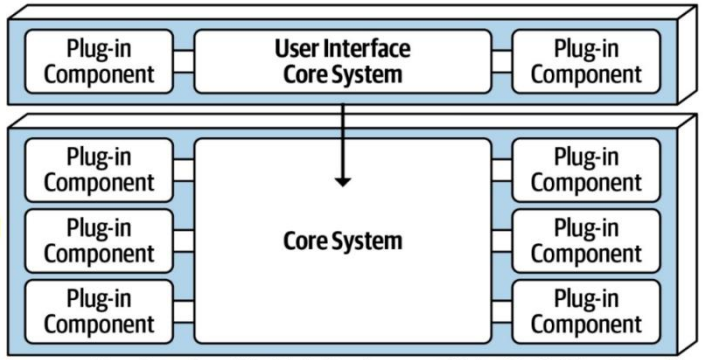
- A separate user interface can also be implemented as a microkernel architecture style.



Embedded User Interface (Single Deployment)



Separate User Interface (Multiple Deployment Units)



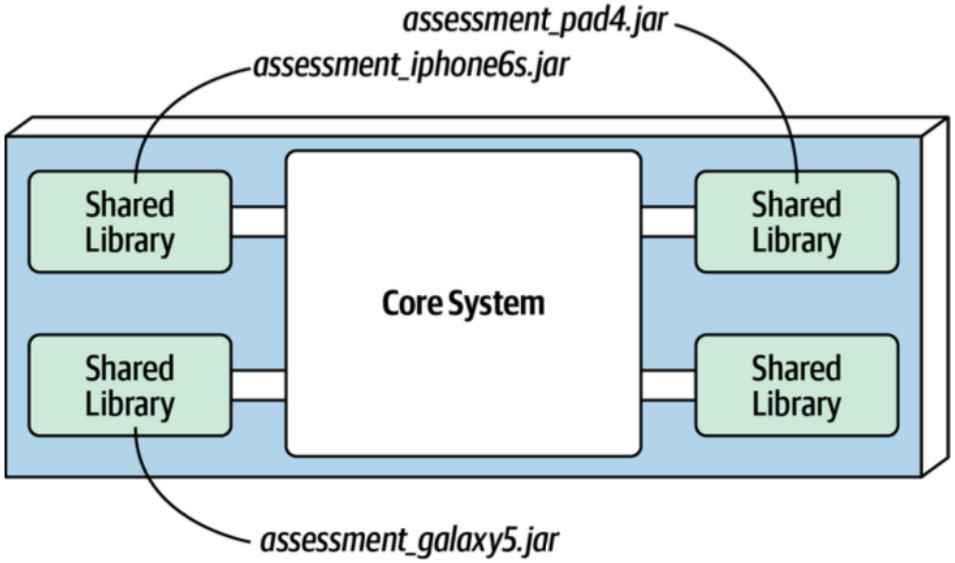
Separate User Interface (Multiple Deployment Units, Both Microkernel)

5.5. Microkernel Architecture Style



Plug-In Components

- Plug-in components are standalone, independent components that contain specialized processing, additional features, and custom code meant to enhance or extend the core system.
- Can be used to isolate highly volatile code, creating better maintainability and testability within the application.
- Plug-in components should be independent of each other.



5.5. Microkernel Architecture Style



Plug-In Components

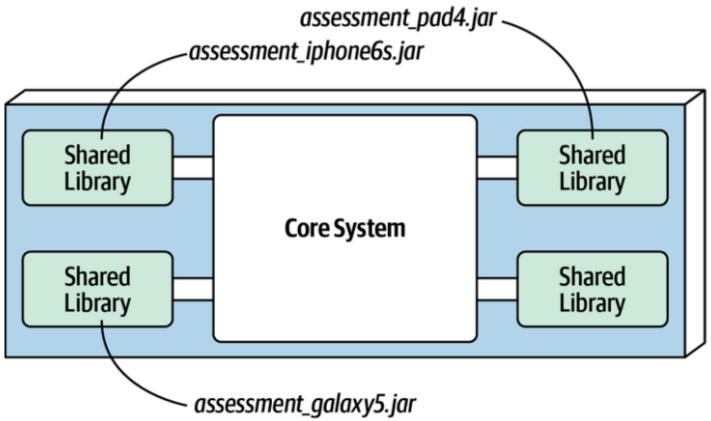
- The communication between the plug-in components and the core system is generally **point-to-point**, meaning the “pipe” that connects the plug-in to the core system is usually a method invocation or function call to the entry-point class of the plug-in component. And can be:
  - Runtime-based: plug-in components can be added or removed at runtime without having to redeploy the core system or other plug-ins.
  - Compile-based: much simpler to manage but require the entire monolithic application to be redeployed when modified, added, or removed.

5.5. Microkernel Architecture Style

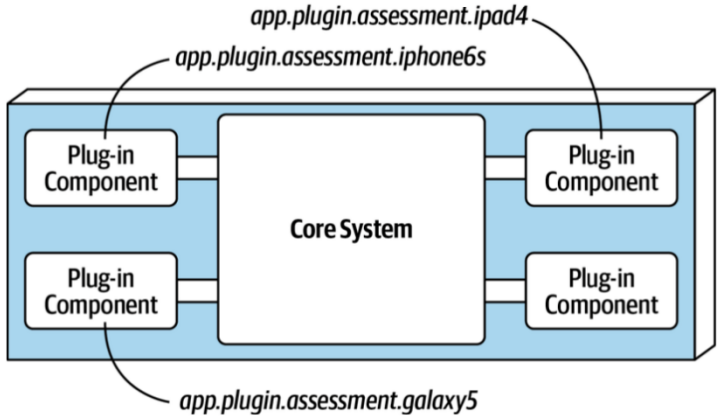


Plug-In Components

- Can be implemented as shared libraries (such as a JAR, DLL, or Gem), package names in Java, or namespaces in C#.
- Runtime plug-in components are usually managed through frameworks such as Open Service Gateway Initiative (OSGi) for Java, Penrose (Java), Jigsaw (Java), or Prism (.NET).



Shared library plug-in implementation



Package or namespace plug-in implementation

5.5. Microkernel Architecture Style



Plug-In Components

Exercises:

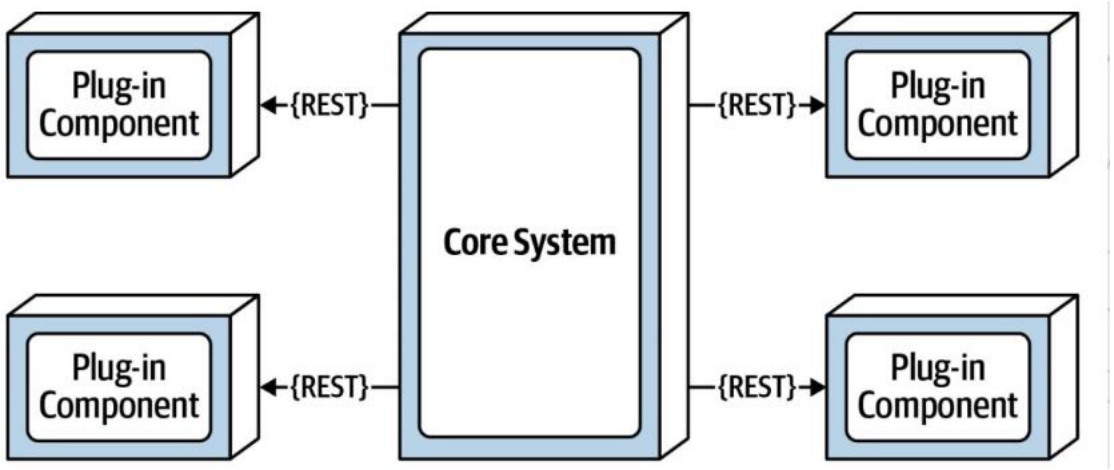
Search and compare frameworks: Open Service Gateway Initiative (OSGi) for Java, Penrose (Java), Jigsaw (Java), or Prism (.NET).

5.5. Microkernel Architecture Style



Plug-In Components

- Other than Point-to-point, plug-in can use REST or messaging
- Better component decoupling, better scalability and throughput, and allows for runtime changes without any special frameworks, asynchronous communications to plug-ins
- Difficult to implement and deploy for most third-party on-prem products. Furthermore, it creates more overall complexity and cost

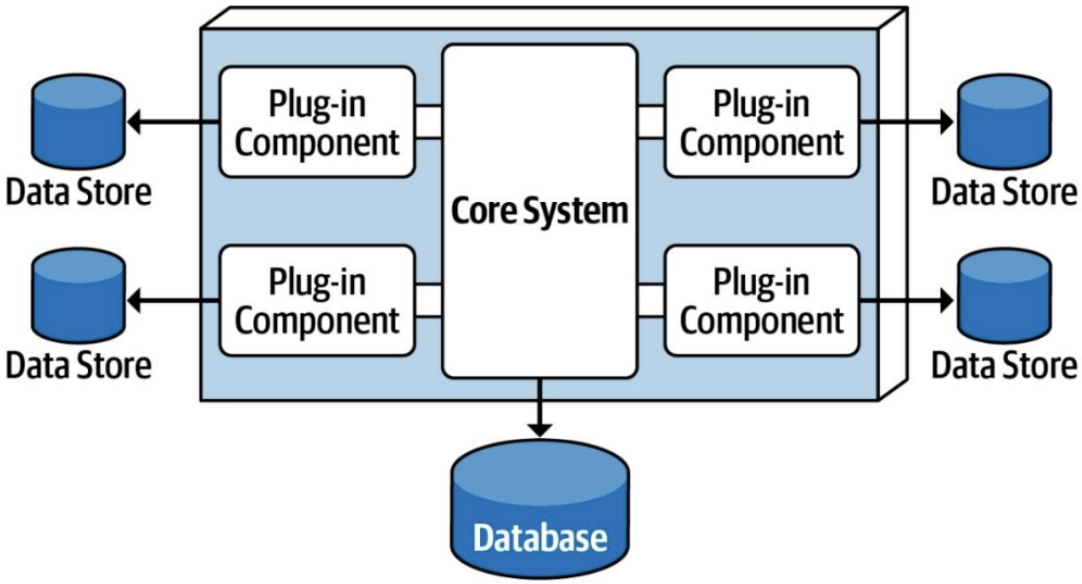


5.5. Microkernel Architecture Style



Plug-In Components

- Central database change should only impact the core system, not the plug-in components.
- Data store owned by the plug-in component can be external.



5.5. Microkernel Architecture Style



Registry

- The core system needs to know about which plug-in modules are available and how to get to them. One common way of implementing this is through a plug-in registry.
- A plug-in registry contains information about each plug-in module. i.e name, data contract, and remote access protocol details.

```
Map<String, String> registry = new HashMap<String, String>();
static {
  //point-to-point access example
  registry.put("iPhone6s", "Iphone6sPlugin");

  //messaging example
  registry.put("iPhone6s", "iphone6s.queue");

  //restful example
  registry.put("iPhone6s", "https://atlas:443/assess/iphone6s");
}
```

A simple registry code with name (key) and access protocol

5.5. Microkernel Architecture Style



Registry

- The registry can be as simple as an internal map structure owned by the core system containing a key and the plug-in component reference.
- or it can be as complex as a registry and discovery tool either embedded within the core system or deployed externally (such as Apache ZooKeeper or Consul).

```
Map<String, String> registry = new HashMap<String, String>();
static {
  //point-to-point access example
  registry.put("iPhone6s", "Iphone6sPlugin");

  //messaging example
  registry.put("iPhone6s", "iphone6s.queue");

  //restful example
  registry.put("iPhone6s", "https://atlas:443/assess/iphone6s");
}
```

A simple registry code with name (key) and access protocol

5.5. Microkernel Architecture Style



Contract

- The contracts between the plug-in components and the core system are usually standard across a domain of plug-in components and include behavior, input data, and output data returned from the plug-in component.
- Plug-in contracts can be implemented in XML, JSON, or even objects passed back and forth between the plug-in and the core system.

```
public interface AssessmentPlugin {
        public AssessmentOutput assess();
        public String register();
        public String deregister();
}

public class AssessmentOutput {
        public String assessmentReport;
        public Boolean resell;
        public Double value;
        public Double resellPrice;
}
```

Contract example

5.5. Microkernel Architecture Style



Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★★
Reliability	★★★
Scalability	★
Simplicity	★★★★
Testability	★★★



Q&A